

Why LLM needed an IR:

What is it and how it works?

By: Ayush Satyam
Associate MLE at  Red Hat

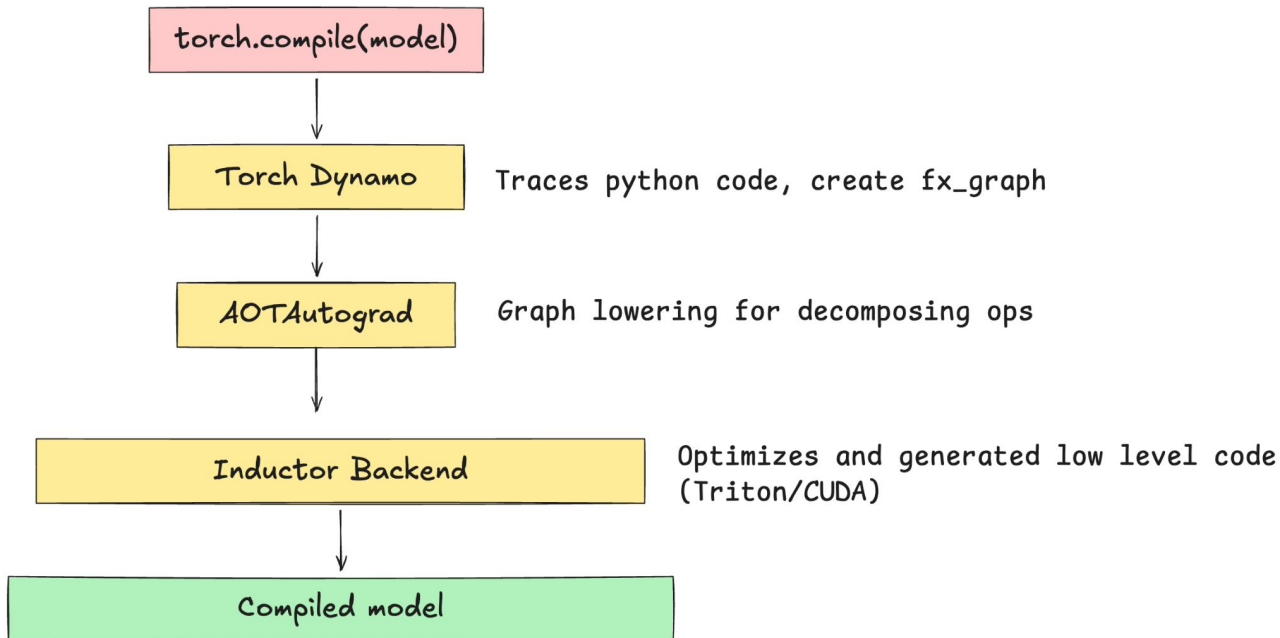
what is vLLM?

vLLM is a fast and easy-to-use library for LLM inference and serving.

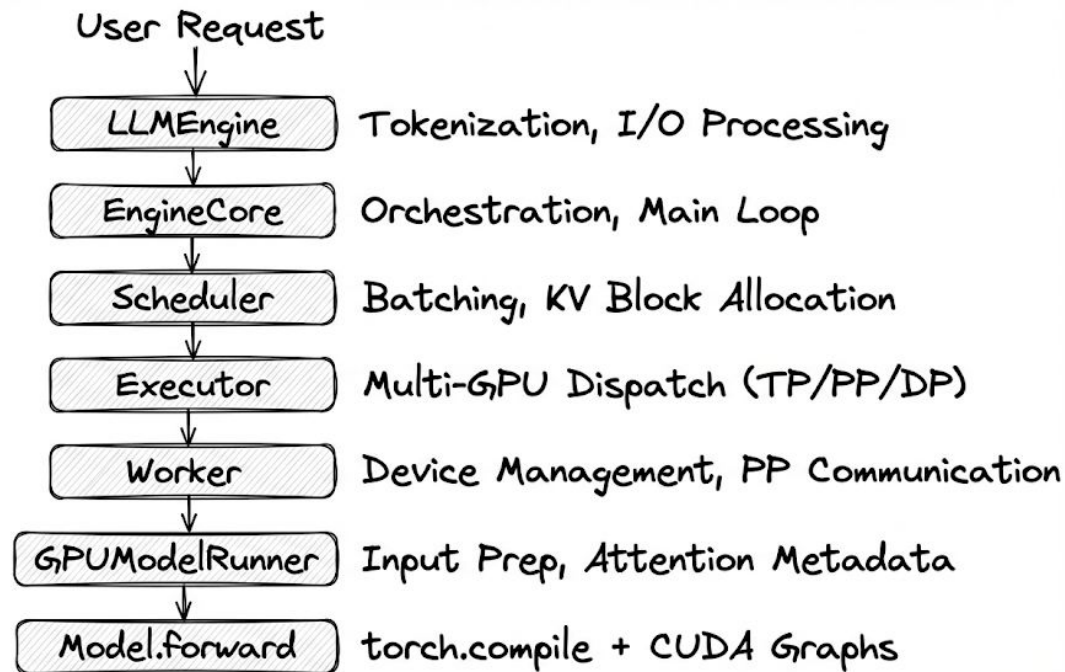
What is **torch.compile**?

torch.compile is a Just-In-Time (JIT) compiler that turns Python **PyTorch code** into highly optimized **kernel code** with a **single line decorator**

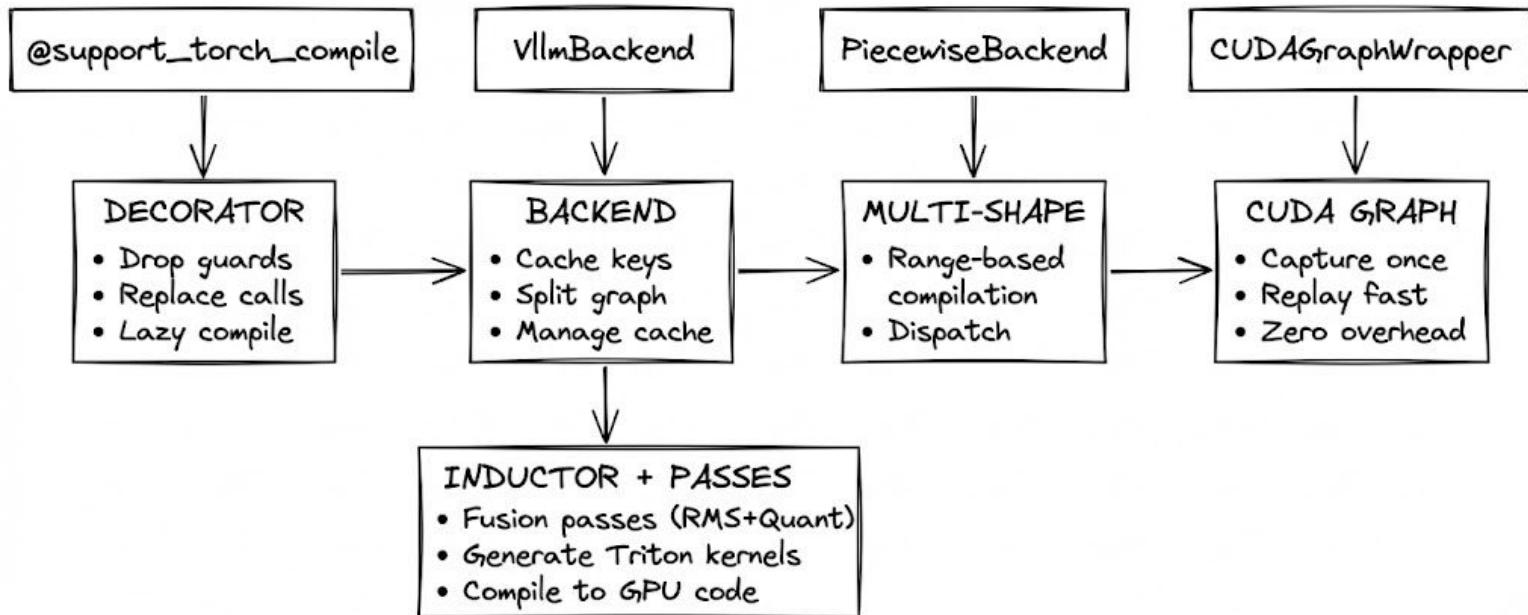
Control flow of **torch.compile**



where torch.compile meets vLLM



VLLM COMPILATION PIPELINE



Where does vLLM IR come from?

[RFC]: vLLM IR: A Functional Intermediate Representation for vLLM

 Open

 Feature

 1 / 2



ProExpertProg opened on Jan 15 · edited by ProExpertProg

Edits ▾ Collaborator ⋮

Summary

This RFC proposes vLLM IR, a functional intermediate representation (IR) based on torch for custom operations like `rms_norm`, `quant`, activations, etc. It addresses issues with compilation and `CustomOp`-based kernel dispatching. More details are available in the [detailed proposal](#). Also take a look at the [slides](#).

Motivation and proposal

Fusion/transformation `torch.compile` passes struggle with custom operators like `RMSNorm`, `Quant`, etc. because they decompose to either a fragile sequence of `torch` ops or a variety of custom kernels, requiring separate handling for each of them. All of those operators are instances of `CustomOp`, which has many of its own issues, mostly stemming from complicated and clunky dispatching logic with low visibility.

vLLM IR is a **functional intermediate representation (IR)** that fills the gap between low-level `torch` ops and vLLM layers like `RMSNorm`, separating the *semantics* from the *implementation* and *dispatching*, solving the issues with `torch.compile` passes and custom op use and dispatching simultaneously. It operates as a **dialect** in the torch FX representation, allowing full interoperability with “regular” torch ops & custom ops, as well as a piecewise migration from the current `CustomOp` approach.

vLLM IR has the following main advantages over the current use of custom ops (& `CustomOp`s):

- Simple and extensible op and implementation registration, in-tree and out-of-tree
- High-level functional compiler IR for easier compilation passes and OOT compilation backends
- Single-source-of-truth dispatching and easier kernel dispatching configuration
- On-demand autotuning of different implementations via `torch.compile`

Why vLLM IR?

Okay....so, vLLM uses torch.compile. But why was that not enough? Why vLLM IR?

The real Questions?

- Why vLLM needed IR beyond CustomOp
- How semantic ops do tracing differently
- How lowering picks backend implementation late!?!?
- How can I use it? What is landed today, what remains open

What is vLLM IR?

vLLM IR is a functional intermediate representation (IR) that fills the gap between low-level torch ops and vLLM layers like **RMSNorm**,

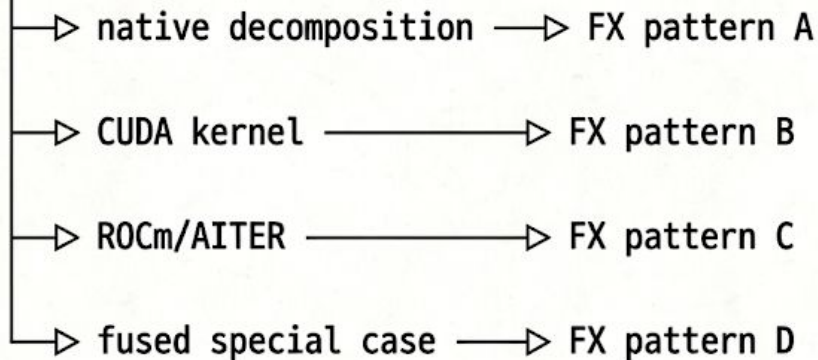
It separates the **semantics** from the **implementation** and **dispatching**, solving the issues with torch.compile passes and custom op use and dispatching simultaneously

Why **CustomOp** Hurt Compiler Ergonomics

- **Mutable custom ops forced awkward auto_functionalized handling**
- **Same op looked different across native and kernel backends**
- **Fusion and pattern matching became fragile**
- **Dispatch logic lived inside stateful layer objects**
- **OOT kernels often replace whole layer/object**

The **Old world** of CustomOps

Layer(CustomOp)



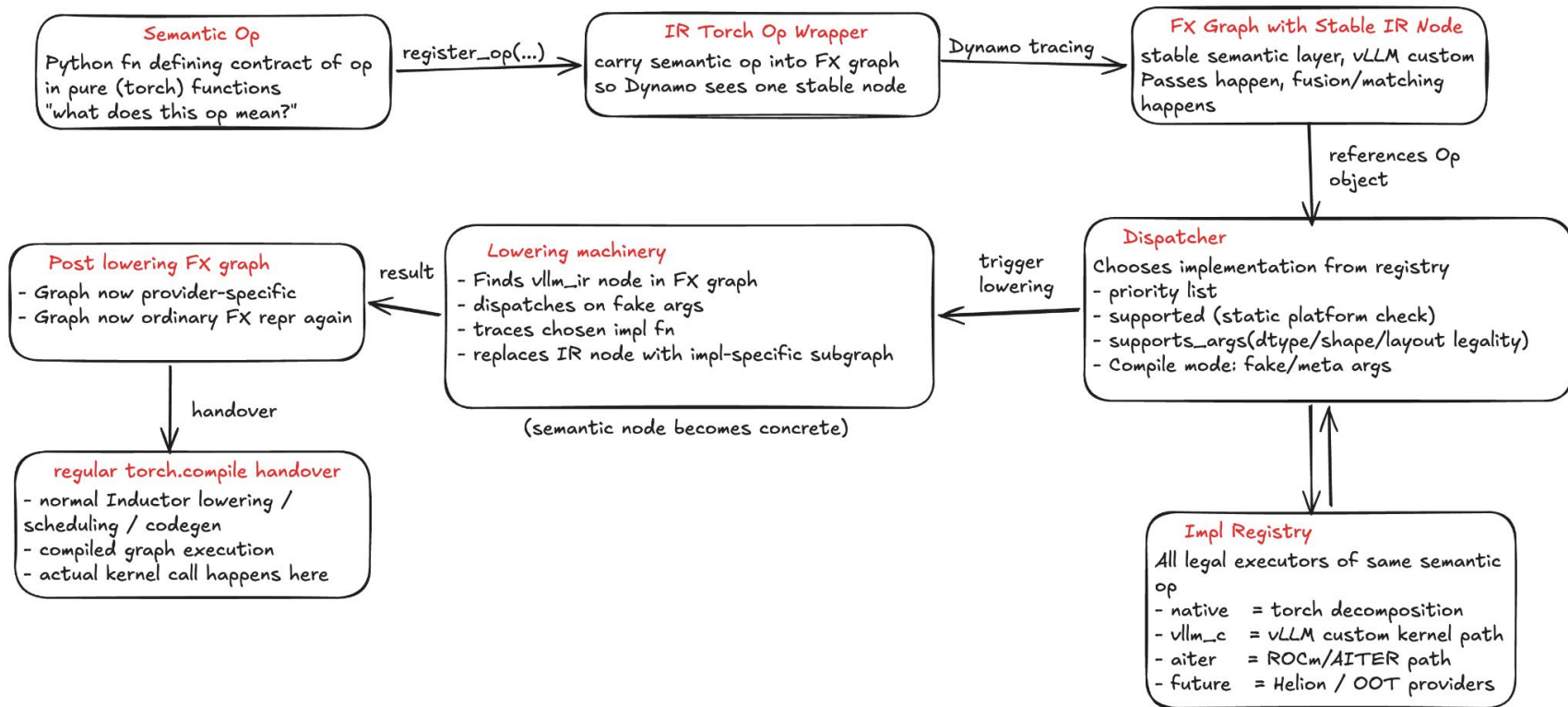
Same meaning, many graph shapes -> compiler pain

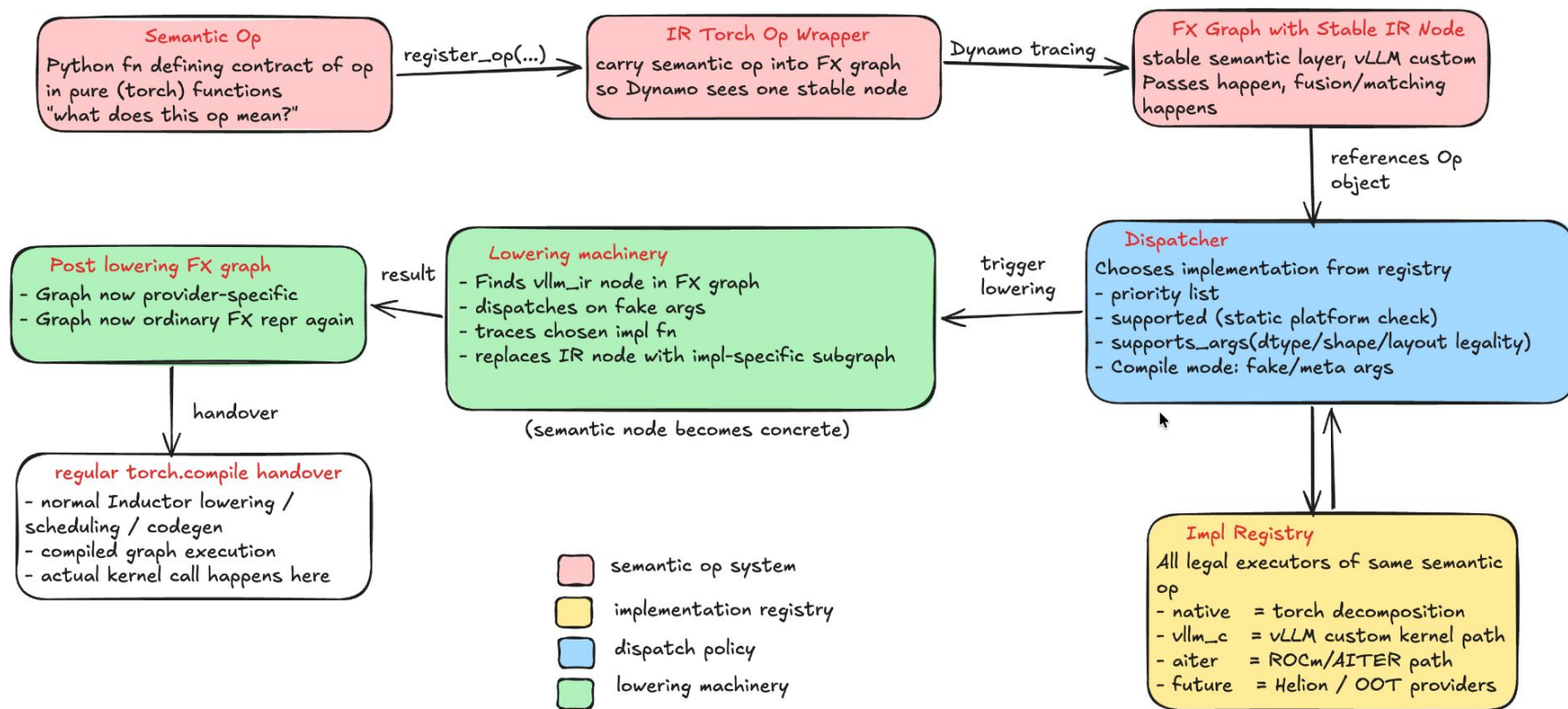
How vLLM IR solves that!

vLLM IR solves that separating **semantics** from
implementation and dispatching

$$\text{vLLM IR} = \begin{array}{c} \text{semantic op system} \\ + \\ \text{implementation registry} \\ + \\ \text{dispatch policy} \\ + \\ \text{lowering machinery} \end{array}$$

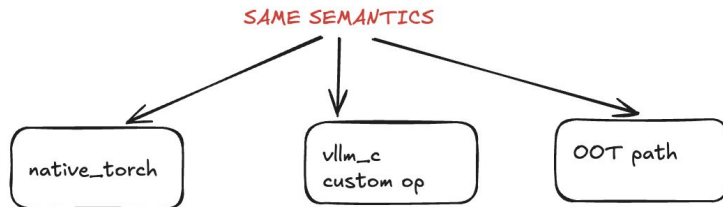
vLLM IR pipeline





Semantic vs Implementation

- One semantic op can have many impls
- Many impls do not imply many **meanings**
- Standardization exists before lowering
- Provider-specific graph appears after lowering



FX graph alterations

BEFORE LOWERING

x --> add --> vllm_ir.rms_norm --> mul --> out

AFTER LOWERING (native)

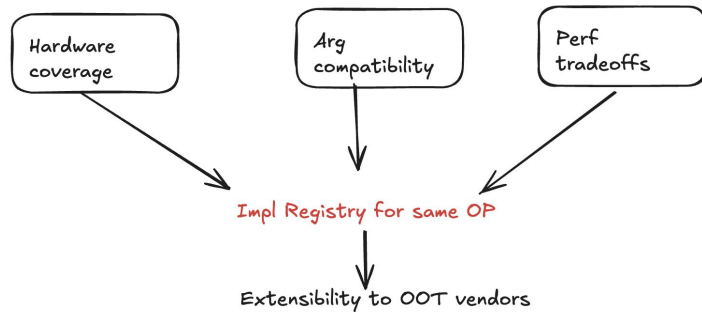
x --> add --> cast --> pow --> mean --> rsqrt --> mul --> cast --> mul --> out

AFTER LOWERING (vllm_c)

x --> add --> empty --> torch.ops._C.rms_norm --> mul --> out

Why **Impl registry** exists?

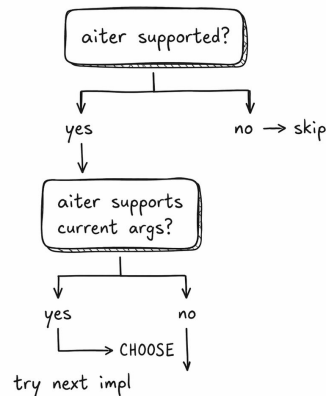
- Different hardware backends
- Different dtype/shape/layout support
- Different perf sweet spots
- Different compile/eager tradeoffs
- OOT platform/provider extensibility



How does **dispatcher** looks?

- Priority list defines preferred impl order
- supported handles static platform availability
- supports_args handles dtype/shape/layout legality
- Compile mode dispatch uses fake args
- Eager mode dispatch uses real args

priority = [aiter, vllm_c, native]



Benefits of vLLM IR to stakeholders

- Kernel devs: easier registration and testing
- Compiler devs: stable semantic nodes for passes
- OOT platforms: clean impl injection path
- Users: clearer config and better kernel selection
- Future: autotuning across implementations

References:

- vLLM IR [RFC Issue](#) (Must read!)
- vLLM [codebase](#)
- vLLM paper [here](#)
- My previous [Talk](#) on vLLM compile

THANK YOU

Reach out to me on:



@ayushsatyam146



@ayushsatyam146

